13/01/26 PCS-253

(Q1) Create a detail with columns Name, Sem, Age, Height, Marks, and study hours.
(i) No. of students in semester
(ii) Outliers by IQR & z-score b/w Marks & study hrs.
(iii) linegraph & comment if it is suitable or not (marks & study hrs).
(iv) Find correlation (marks and study hrs.
(v) Fill all null values

Sol:-

(i)
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

d = {
"name": ["Ansh", "Tamanna", "Jiya", "Aarzoo", "Girija"],
"sem": [2, 2, 3, 2, 3],
"age": [19, 18, np.nan, 19, 22],
"height": [170, 165, 175, np.nan, 172],
"marks": [78, 85, 92, 60, np.nan],
"studyhrs": [4, 6, np.nan, 8, 6]
}
df = pd.DataFrame(d)
print(df)
```

```
df = pd.DataFrame()
print(df)
```

(i)
```
# No. of students in each sem
print(df['sem'].value_count())
```

```
print(df['sem'].value_count())
```

(ii)
```
# Fill all null values
df_filled = df.fillna(df.mean(numeric_only=True))
print(df_filled)
```

```
df.fillna(df.mean(numeric_only=True))
```

(iii) # outliers using IQR

```
q1 = df['marks'].quantile(0.25)
q3 = df['marks'].quantile(0.75)
iqr = q3 - q1
upp = q3 + 1.5 * (iqr)
print(upp)
low = q1 - 1.5 * (iqr)
print(low)
df = df[(df['marks'] >= low) & (df['marks'] <= upp)]
print(df)
```

#So that fluctuation error does not occur, outliers value are required.

(iv) Outliers using Z-score

```
Z-marks = np.abs((df["marks"] - df["marks"].mean()) / df["marks"].std())
Z-hrs = np.abs((df['study-hrs'] - df['study-hrs'].mean()) / df['study-hrs'].std())
print(df['marks'][Z_marks > 3]) #outliers in marks
print(df['study hrs'][Z-hrs > 3]) #outliers in study hrs
```

(2.5)

(v)

```
df_sorted = df.sort_values("study_hrs")
plt.plot(df_sorted['study_hrs'], df_sorted['marks'], marker='o')
plt.xlabel('Study hrs')
plt.ylabel("Marks")
plt.title("line graph: Marks vs study Hours")
plt.show()
```

(v) correlation = df_filled['marks'].corr(df_filled['studytime'])
print("correlation")

2(0) Import a csv file 22yztw(medical_records.csv) and do all data cleaning operations on it.

3(0) Import a csv file 6svf01.csv (patient-details.csv) and do all data cleaning operations on it

3(0) Merge both cleaned datasets on the basic of patient_id column.

Data Cleaning and Merging Script

1. Import and clean First Dataset
We'll start with medical_record.csv. "Cleaning" usually involves handling missing values, removing duplicates, or fixing data types

```
import pandas as pd
# Load the first dataset
df1 = pd.read_csv('medical_recorlds.csv')
# General cleaning operations
df1.drop_duplicates(inplace=True)
df1.dropna(inplace=True) # Or use df1.fillna() depending on your needs
```

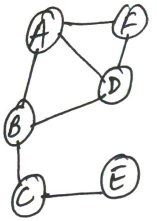
2. Import and clean second dataset

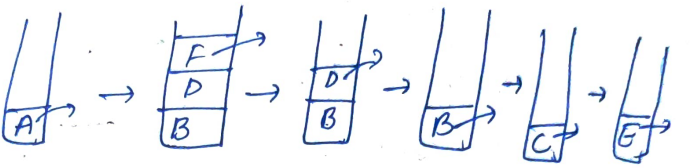
```
df2 = pd.read_csv('patient_details.csv')
df2.drop_duplicates(inplace=True)
df2.dropna(inplace=True)
```

3.
```
merge_df = pd.merge(df1, df2, on='patient_id', how='inner')
print(merged_df.head())
```

- Import → Reading .csv files into Pandas Data frames.
- clean → Removing noise like duplicates or empty rows.
- Merge → linking records where patient_id matches in both files.

#



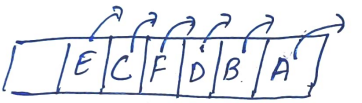


Visited :- A B D F C E

Resultant :- A F D B C E

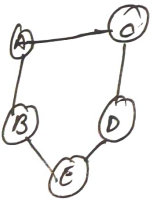
→ DFS.

BFS :-

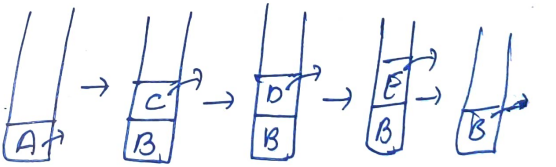


Visited :- A B D F C E

Resultant :- A B D F C E ✓



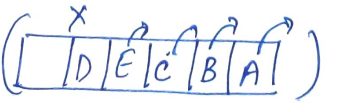
DFS :-



Visited :- A B C D E

Resultant :- A C D E ✓

BFS :-



Visited :- A B C E D

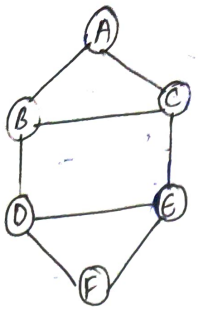
Resultant → A B C E ~~X~~

Resultant → A B E

Visited :- ~~A B E~~ A C B E D.

J-02-2020

(Q3)



A-B
A-C
B-D
C-E
E-F
D-F
B-C
D-E

- Denote :-
  - ↳ parent → U
  - ↳ child → V

Write code for DFS for the given graph.

Ans:-

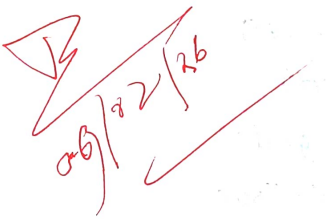
```
def dfs(start, goal, graph, vis=None):
    if vis is None:
        vis = set()
    print(start)
    if start == goal:
        return True
    vis.add(start)
    for child in graph[start]:
        if child not in vis:
            if dfs(child, goal, graph, vis):
                return True

graph = {}
n = int(input("Enter no. of nodes="))
for i in range(n):
    k = input("enter node:")
    graph[k] = []
    m = int(input("how many neighbours:"))
    for in in range(m):
        v = input("enter neighbours:)
        graph[k].append(v)
```

```
graph
S = input("enter starting point:")
E = input("enter ending point:")
print(dfs(S,E,graph))
```

OUTPUT

```
Enter no of nodes = 6
 enter node: A
how many neighbours: 2.
 enter neighbour: B
 enter neighbour: 8
 enter node: B
 how many neighbours: 3
  enter neighbour: A
  enter neighbour: B
  enter neighbour: E
  enter Node: D
 how many neighbours: 3
  enter neighbour: B
  enter neighbour: E
  enter neighbour: F
   enter node: E
 how many neighbour: 3
  enter neighbour: C
  enter neighbour: D
  enter neighbour: F
  enter node: F
  how many: 2
   neighbour
  enter neighbour: D
 enter neighbour: E
```



```
enter starting pt: A
enter ending pt: F
A
B
C
E
D
F
True
```

What is a graph and why do we require it?

Ans:-

Types of graph

What is depth for search (DFS)?

Any any 5 real life application of DFS and how it used

What is breath for search (BFS) and write its any 5 real life application

Differences between BFS & DFS.

0# Create a data set for medical insurance which has column name as, Id, Name, Age, ~~diagnosis~~, sugar level, BP, income, ~~occupation~~ of the patient. Normalize age, sugar, and income

Why are you using normalisation?

↳ Normalisation is a process of scaling data to a specific range usually 0 to 1:

Without Normalization,

- Large scale features dominate
- Small-scale features ignored.

outliers :- Data point significantly differs from other observations.

- A graph is a non-linear data structure consisting of
  - Vertices → (Nodes)
  - Edges → (links).

Mathematical representation → $G = VE$

V → set of Vertices, E → set of Edges

Why we use graphs:

1) To represent real-world networks
2) Used in network routing and AI problems.
3) Used in path finding and navigation

Types of graph:-

(1) Cyclic
(2) Acyclic
(3) Connected
(4) Complete
(5) Disconnected
(6) Bipartile Graph
(7) Undirected
(8) Directed

(2) DFS is a graph traversed algorithm used to visit all nodes of a graph or tree. It starts from a source nodes & explores as far as possible along each branch before back tracking

Working Principle:-

→ Starts from root/starting node
→ Visit an adjacent unvisited node
→ Continue visiting deeper nodes.
→ If no visited is left → Backtrack.
→ Repeat until all nodes are visited

DFS is implemented using → STACK(IFDOor recursion)

class

(1) Helps find a path between nodes.
(2) Detects loops in graph.
(3) Used in task scheduling → Topological sorting
(4) Finds exist path
(5) checks connecting.

5 → Real world Examples :-

(1) File system reading
(2) Cycle detection in Network
(3) Web Graveling
(4) Maze solving
(5) Social Network Analysis.

(3) ⁂ BFS :- Breadth first search is a graph transversal algorithm that visits nodes level by level. It first visits all neighbouring nodes, then moves to next level neighbours.

Working principle :-

(1) Start from a source code.
(2) Visit all adjacent node first.
(3) Then visit neighbours of all those nodes.
(4) Continue until all nodes are visited.

BFS → Queue FIFO)

Why use it :-

(1) To find shortest path in unweighted graph.
(2) Used in AI search problem.
(3) Helps in peer to peer networks

Real Life Examples:-

(1) Network Broadcasting
(2) Social Media
(3) Web Crawlers
(4) GPS Navigation
(5) Finding Network devices

Differentiate b/w DFS & BFS:-

basis	BFS	DFS.
Traversal style	Level by level	Goes deep into branches first.
Data Structure used	Queue	Stack
Shortest path	Finds shortest path in unweighted graph	Does not guarantee shortest path
Memory	Higher	Low
Speed	Slower for deep search	Faster
Application	Shortest path	Cycle detection

(4) Write a A* algorithm code and make a graph and calculate the path.

→
```
import networkx as nx
graph = nx.Graph()
while True:
ch = int(input('want to insert node & enter edges
        (press for yes & 0 for no:"))
if ch == 1:
node1 = input("enter node 1").strip().upper()
node2 = input("enter node 2").strip().upper()
node3 = input("enter nod 3").strip().upper()
wt = int(input("enter wt b/w node 1&2"))
graph.add_edge(node1, node2, weight=wt)
else:
break
heuristic={}
nodes = list(graph.nodes())
print("enter heuristic value for each other')
for node in nodes:
heuristic[node] = int(input(f'h{node}) = '))
def h(n1, n2):
return heuristic[n1]
st = input("start node").strip().upper()
go = input("goal node").strip().upper()
```

```
path = nx.a_star_path(graph, st, heuristic = h1,
                      weight = weight)
print("a* path:", path)
```

Output:-

Want to insert & enter edges (press 1 for yes & 2 for no:)

Enter node 1: a

Enter node 2: b

Enter the weight b/w node 1 & node 2: 4

Want to insert and enter edges (press 1 for yes & 2 for no): 1

Enter node 1: b

Enter node 2: c

Enter the weight b/w node 1 & node 2: 5

Want to insert & enter edges (press 1 for yes & 2 for no): 2

Enter heuristic values for each node.

Enter h(A): 3

Enter h(B): 5

Enter h(C): 2

Start node: a

goal ~~start~~ node: c

A* path: ['A', 'C']

→ For kNN :- And SVM

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.neighbours import kNeighbourClassifier
from sklearn.model_selection import train_test_split
from sklearn.matrices import mean_squared_error, mean_absolute_error, r2_score, accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
from sk.learn.preprocessing import LabelEncoder,
```

→
```
data = {
    "p_id" = np.random.randint(1,5000, 5000),
    "disease" = np.random.choice(["Yes", "NO"], 5000)
}
```

→
```
df = pd.DataFrame(data)
df
```

→
```
df.duplicated().sum()        # → output → 1086
x = df["p_id"]
y = df[["disease"]]
```

→
```
x, y
```

→
```
le = LabelEncoder()
df["disease_lb"] = le.fit_transform(df["disease"])
df
```

x_test, y_train

* x_train, x_test, y_test = train_test_split (x, df ["disease_no"],
  test_size = 0.2, random_state = 42)

* knn = KNeighborsClassifier (n_neighbors = 13)

* knn.fit (x_train, y_train)

* y_pred = knn.predict (x_test)

* y_test

* y_pred

* accuracy_score (y_test, y_pred)

* precision_score (y_test, y_pred)

* recall_score (y_test, y_pred)

* f1_score (y_test, y_pred)

* print (confusion_matrix (y_test, y_pred))

* data = {
  "p_id": np.random.randint (1, 5000, 5000),
  "age": np.random.randint (15, 80, 5000),
  "weight": np.random.randint (35, 80, 5000),
  "disease": np.random.randint (["BP", "Diabetes", "Cancer", "Asthama",
  "Jaundice"], 5000)
  }

df = pd.DataFrame (data)
df

* x = df [["p_id"]]
  y = df ["disease"].

1(Q) Implement random function to generate marks of 380 students using "random.randint()".

2(Q) Implement A-star

3(Q) Implement BFS.

(1) import numpy as np
import random
n_data = np.randint.(1, 101, size = 30)
marks = list(n_data).
print(marks)

#output ⇒ [np.int32(37). np.int32(59)---np.int32(83)].

Soln(2) #A*code
. import networkx as nwx
graph = nwx.Graph()
graph
while True:
choice = int(input("want to insert nodes and enter edges? 1 for yes and 2 for no))

if choice == 1;
node1 = input("Enter Node1:").strip().upper()
node2 = input("Enter node 2:").strip().upper.
weight = int(input("Enter weight b/w node1 and node2:"))
graph.add_edge(node1, node2, weight=weight)

else:
break
graph.adj.items()
for node, neighbours in graph.adj.items():
Print(node, neighbours).

#output:-

A {'B': {'weight': 5}, 'C': {'weight': 3}}
B {'A': {'weight': 5}, 'C': {'weight': 4}}
C {'A': {'weight': 3}, '3': {'weight': 4}}

```
start = input("Enter start node: ").strip().upper()
goal = input("Enter goal node:").strip().upper()
```

#output → Enter start node: a
Enter goal node: c

```
graph.nodes()
```

output => NodeView(('A', 'B', 'C'))

```
heuristic_values = {}
heuristic_values[goal] = 0.
for node in graph.nodes():
    if node != goal:
heuristic_value :- int(input(f"Enter heuristic value from
                    node {node} to goal node {goal}: "))
    heuristic_value[node] = heuristic value
heuristic_values
```

Thanks

Abhay
25-9-26

output :-

{'C': 0, 'A': 2, 'B': 3}

```
def get_heuristic values [u]
path = mwx.astar_path (graph, start, goal, node heuristic = get_
                       heuristic ; weight = 'weight") → ".join(path)

#output → 'A' → 'C'
total_cost = mwx.astar_path_length (graph, start, goal, heuristic,
           = get_heuristic, weight = 'weight')

print(f" total_cost}\n")

#output → 3
```

Sol 3: #BFS code:-

```
d = { }
n = int (input ("Enter total neigh:"))
for i in range(n):
u = input (f" Enter node {i}:")
  d[u] = [ ].
n = int (input("Enter num of neigh at {i}:"))
for i in range(n):
  v = input()
d[u].append (v)
print (d).
```

```
from collections import deque
def bfs(graph, start_node):
    vis = set()
    queue = deque([start_node])
    vis.add(start_node)
    while queue:
        node = queue.popleft()
        print(node, end=" ")
        for neigh in graph.[node]:
            if neigh not in vis:
                vis.add(neigh)
                queue.append(neigh).
```

(Q) Create a medical dataset which has patient_id, age, data, sugar_level, bp, income of patient, gender, height, & diagnosis then calculate normalization on height, income, age, sugar_level, bp:-

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
d = pd.read_csv("patient_data.csv")
df = pd.DataFrame(d)
print(df)
```

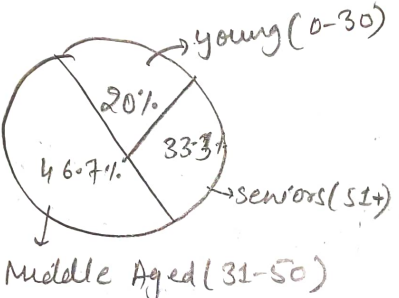
#output =

	Patient Id	age	data	sugar level	bp	income	height	gender
0	P101	45	2006-1-10	110	120	55000	175	M
1	P102	32	2020-2-12	145	185	42000	162	F
2	P108	29	2026-01-25	2000	115	5300	160	F.

```
bins = [0, 30, 50, 100]
labels = ['young(0-30)', 'Middle-aged(31-50)', 'senior(57+)']
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels)
data = df['age_group'].value_counts()
colours = sns.color_palette('pastel')[0:len(data)]

#Pie-chart:-
plt.pie(data, labels=data.index, colors=colors, autopct='%.1.1f%%')
plt.title('Distribution of Patients by Age group')
plt.axis('equal')
plt.show()
```

output



Distribution of Patients by age group.

```
cols_to_norm = ['age', 'height', 'sugar_level', 'bp', 'name']
for col in col_to_norm:
df[f'norm_{col}'] = (df[col] - df[col].min())/(df[col].max() - df[col].min
print(df['age', norm_age', 'height', 'norm_height', 'sugar_level', 'norm_
  sugar_level', 'norm_income'])
```

output:-

	age	norm_age	height	norm_height	income	norm_income
0	24	0.000	175	0.7407	55000	0.0500
1	73	1.0000	158	0.1111	82000	1.000
2	55	0.6326	182	1.000	28000	0.00

Graphs of Normalized data:-

```
fig, axs = plt.subplots(2, 2, figsize=(10,8))
fig.suptitle('Normalized metrices per patient')

# Normalized ~~Size~~ Age
axs[0,0].plot(df.index, df['norm_age'], marker='O', color='blue')
axs[0,0].set_title('Normalized age')
axs[0,0].set_ylabel('Normalized value')

# Normalized BP
axs[0,1].plot(df.index, df['norm_bp'], marker='O', color='green')
axs[0,1].set_title

# Normalized sugar_level
axs[1,0].plot(df.index, df['norm_sugar_level'], marker='O', color='red')
axs[1,0].set_title('Normalized Sugar level')
axs[1,0].set_ylabel('Normalized value').

# Normalized Income
axs[1,1].plot(df.index, df['norm_income'], marker='O', color='yellow')
axs[1,1].set_title('Normalized Income')
```

for SVM

```
np.random.linespace(4,100,(1000))
np.random.choice(['a','b','c'],(1000))
np.random.randint((500,400),200)
np.random.uniform(500,400,200)
np.random.normal(500,400,200)
```

(Q) Perform a linear regression on the dataset.

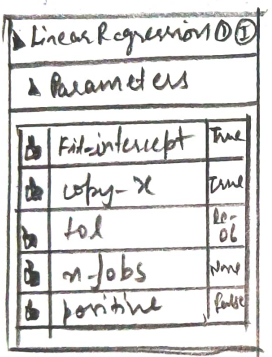
```
import pandas as pd.
import numpy as np
import matplotlib.pyplot as plt

data = {
'name'= ['std.1', 'std.2', 'std.3', 'std.4', 'std.5', 'std.6', 'std.7']
'age' = [19,28,17,32, 23, 18,22]
'rollno': [1,2,3,4,5,6,7]
'marks': [98,94,90, 76,82,60,85]

df1 = pd.dataframe(data)
df1.to_csv("data1.csv")
df = pd.read_csv('data1.csv')
plt.scatter(df['rollno'], df['age'])
plt.xlabel('Roll No')
plt.ylabel('age')
plt.title('Roll No. Vs Age')
plt.show()
x = df['age']
y = df['rollno']
x = np.array(x).reshape(-1,1)
y = np.array(y).reshape(-1,1)
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
x_train, x_test, y_train, y_test = train_test_split(x, y,
        test_size = 0.2, random_state = 10)
model = linear.Regression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
y_pred.
```

outputs :-

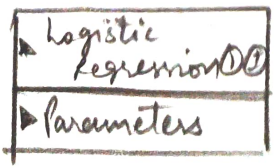


array([[4.6266307], [4.66414687]))

#Perform a logistic regression code:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, classification_report
from sklearn.linear_model import LogisticRegression
np.random.seed(42)

n = 8
exp = np.random.uniform(0,20,n)
sa = 3000 + (exp * 5000) + np.random.normal(0,5000,n)
df = pd.Dataframe(
  {'Years of experience': exp,
   'salary': sa})
df.to_csv('salary-prediction.csv', index=False)
x = df[['Years-of-experience']]
y = df['salary']
x_train, x_test, y_train, y_test = train_test_split(x,y,test_
size=0.2, random_state=12)
lr = LogisticRegression()
lr.fit(x_train, y_train)
```



```
y_pred = lr.predict(x_test)
as = accuracy_score(y_test, y_pred)
ps = precision_score(y_test, y_pred)
rs = recall_score(y_test, y_pred)
fs = f1_score(y_test, y_pred)
print(confusion_matrix(y_test, y_pred))
```

```
[[1,0],
 [0,1]]
```

```
print(classification_report(y_test, y_pred))
```

	Precision	Recall	f1-score	Support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

(9) Perform K Mean Code :-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import kmeans
From sklearn.preprocessing import standard scaler
x,y = make_blobs(n_samples = 200, centres = 3,
                 random_state = 12)
df = pd.DataFrame(x, columns = ['feature1', 'feature2']).
```

```
ss = StandardScaler()
x_scaled = ss.fit_transform(df)
wcss = []
for i in range(1,8):
    km = KMeans(n_cluster = i, random_state = 12)
    km.fit(x_scaled)
    wcss.append(km.inertia_)
print("\n wcss values for different k:")
for i in range(len(wcss)):
print(f"K={i+1} → wcss ={wcss(i)}")
plt.plot(range(1,8), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('K')
plt.ylabel('wcss')
plt.show()
km = KMeans(n_clusters = 3, random_state = 12)
y_km = km.fit_predict(x_scaled)
print("\n Cluster Centres:")
print(km.cluster_centres)
print("\n cluster labels(first 10 points):")
print(y_km[:10])
```

output:-

wcss values for different K:-
K=1 → wcss → 400.27
K=2 → wcss → 210.45
K=3 → wcss → 85.67
K=4 → wcss → 70.12
K=5 → wcss → 60.34
K=6 → wcss → 52.89
K=7 → wcss → 48.21.

Cluster Centres :-

```
[[-1.2.   1.1]
 [0.9   -0.8]
 [1.5    1.3].]
```

cluster labels (first 10 points):

```
[1 1 0 1 0 2 2 0 1 2]
```

(Q) Perform Random forest Classifier Code:-

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_
matrix, classification_report
data = load_iris()
x = data1.data
y = data.target
df = pd.Dataframe(x, columns = data1.feature_names)
df['target'] = y
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size = 0.3, random_state = 42)
model = RandomForestClassifier(n_estimators = 100,
random_state = 1)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
```

▲ Random forest classifier ① ②

▲ Parameters:

```
Print ('\n Accuracy:')
Print (accuracy_score (y_test, y_pred))
print ('\n Confusion Matrix:')
Print (confusion_matrix (y_test, y_pred))
print ('\n classification Report:')
Print (classification_report (y_test, y_pred)
```

Accuracy:-

1.0

Confusion Matrix:

```
[[13  0  0
  0 13  0
  0  0 13]]
```

Classification Report:-

Precision	recall	f1-score	support
0	1.00	1.00	1.00
1	1.00	1.00	1.00
2	1.00	1.00	1.00
		accuracy	1.00

Abhinav
29-09-24